Global Disaster Events Analysis

Final Project Report — Introduction to Big Data

Adventist University of Central Africa — Department of Information Technology

Name: NKURANGA BAZIGA CALEB

Student ID: 28845

Lecturer: KAYITARE Elie

TABLE OF CONTENT

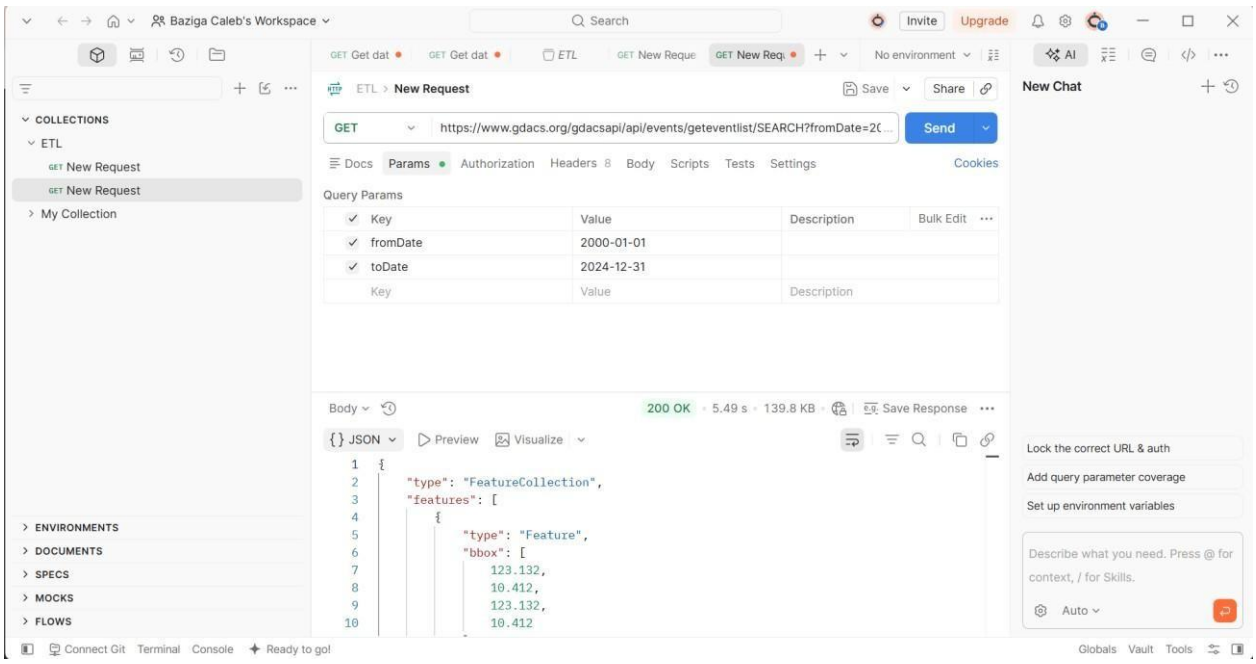
1. Overview: Dataset and Question

For this project I used disaster data from GDACS (Global Disaster Alert and Coordination System). GDACS tracks natural disasters like earthquakes, floods, cyclones, droughts, volcanoes and wildfires from all over the world. I picked this dataset because disaster data has real geography, clear categories, and enough numeric detail to build both a dashboard and machine learning models.

The question I wanted to answer was: given basic facts about a disaster event (its type, how severe it was, and where it happened), can we predict whether GDACS will mark it as an Orange alert (moderate) or a Red alert (severe)? This is useful because it shows whether the size and location of an event on its own can hint at how serious GDACS will judge it to be.

- Data source: GDACS SEARCH API (https://www.gdacs.org/gdacsapi/api/events/geteventlist/SEARCH)
- Time range: 2000 to 2024
- Event types included: Earthquakes (EQ), Floods (FL), Tropical Cyclones (TC), Droughts (DR), Volcanoes (VO), Wildfires (WF)
- Final cleaned dataset size: 1,770 disaster events

I chose to only keep Orange and Red alert events. GDACS's Green alert level (minor events) is left out of its SEARCH endpoint by default. I tested turning it on and got over 10,000 extra events for the same date range, which showed that Green events make up the huge majority of GDACS's data. Since Green events are minor and would have made the dataset much bigger without adding much analytical value, I decided to keep the project scoped to Orange and Red events only, which are the ones that matter most for understanding serious disasters.

- 

2. Data Pipeline: Extraction, Transformation, Loading

2.1 Extraction

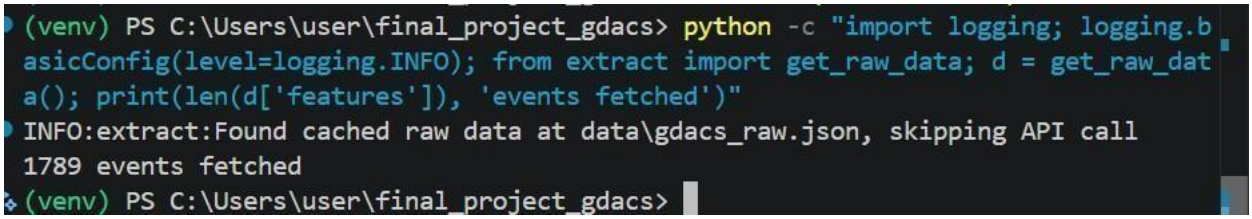
The pipeline downloads data from the GDACS API using Python's requests library. Two problems came up during this step that needed real fixes, not just theoretical ones:

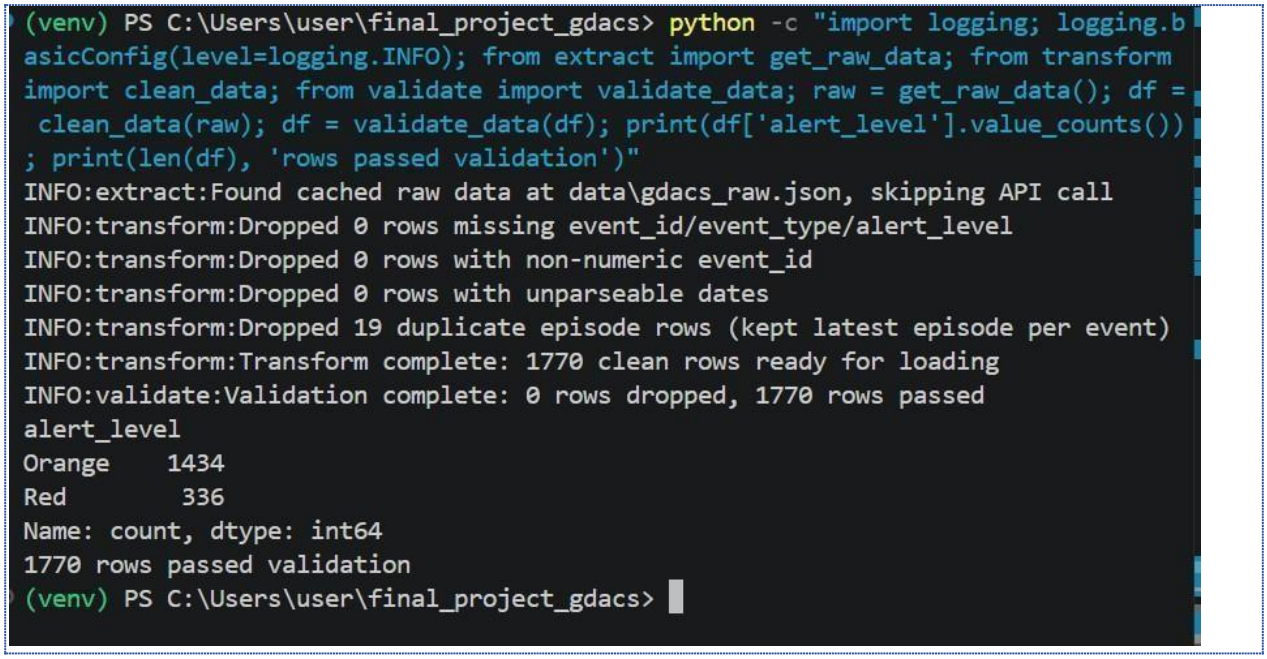
The API only returns 100 records per request. My first attempt looked complete but was actually silently cut off. I fixed this by adding a loop that keeps requesting the next page until a page comes back with fewer than 100 records, meaning there is nothing left to fetch.

- If a request fails (for example a short network problem), the pipeline retries up to 3 times, waiting longer each time (1 second, then 2, then 4) before giving up with a clear error message.

To avoid downloading the same data twice, the raw API response is saved to a local file (data/gdacs_raw.json) after the first successful pull. If the pipeline is restarted, it checks for this file first and skips calling the API again unless a fresh pull is asked for.

-





2.2 Transformation

The transform step turns GDACS's nested JSON data into a flat, clean table. A few cleaning decisions were made, based on problems I actually found in the data:

- GDACS reports the same disaster multiple times as new information comes in (called episodes). I kept only the latest episode for each event, so each real disaster is counted once, not several times.
- Many smaller events have no name in GDACS's data. This is expected and not an error, so I kept these rows with a blank event name instead of deleting them.

  I originally planned to use population exposed to the disaster as a feature. After checking the raw API data directly, I found that GDACS's SEARCH endpoint never includes this field for any event type. I removed it from the project rather than pretend it was available.
- During testing, I found that some missing text values were being saved into the database as the literal word "NaN" instead of being properly empty. I traced this to how pandas and the database library handled missing values together, and fixed it so missing text is now stored correctly as empty (NULL).

-
- I also found that some country names had extra blank spaces (for example "China " with a space at the end, separate from plain "China"). This would have caused Tableau to treat them as two different countries. I fixed this by trimming extra spaces from country names during cleaning.

These were genuine bugs I found by checking the actual exported data rather than assuming the pipeline was correct, and each one is a small example of why checking your own output matters in a real data project.

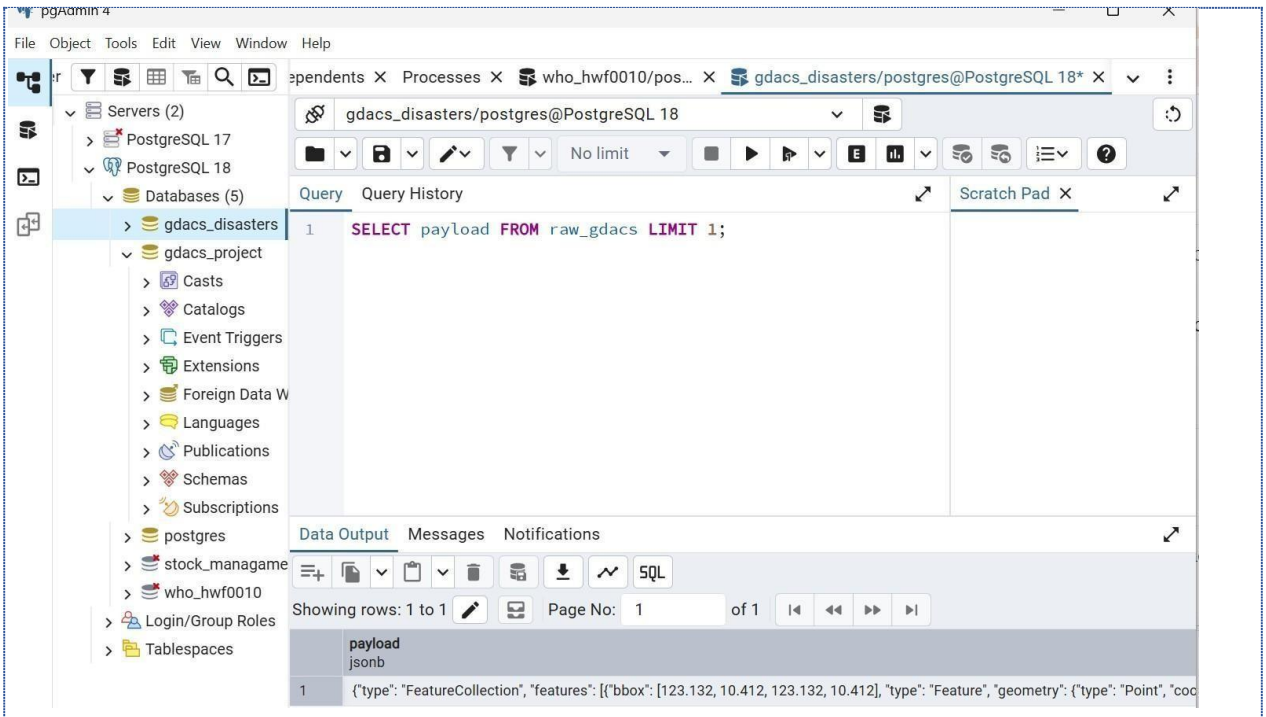
```
(venv) PS C:\Users\user\final_project_gdacs> python -c "import logging; logging.b
asicConfig(level=logging.INFO); from extract import get_raw_data; from transform
import clean_data; raw = get_raw_data(); df = clean_data(raw); print(df.head(10))
; print(len(df), 'clean rows')"
INFO:extract:Found cached raw data at data\gdacs_raw.json, skipping API call
INFO:transform:Dropped 0 rows missing event_id/event_type/alert_level
INFO:transform:Dropped 0 rows with non-numeric event_id
INFO:transform:Dropped 0 rows with unparseable dates
INFO:transform:Dropped 19 duplicate episode rows (kept latest episode per event)
INFO:transform:Transform complete: 1770 clean rows ready for loading
   event_id  episode_id event_type  ... severity_value severity_unit  year
0      1655           1         FL  ...           4.45           NaN  2000
1      1660           1         FL  ...           4.25           NaN  2000
2      1672           1         FL  ...           5.75           NaN  2000
3      1669           1         FL  ...           4.58           NaN  2000
4      1671           1         FL  ...           6.58           NaN  2000
5      1681           1         FL  ...           4.84           NaN  2000
6      1675           1         FL  ...           5.77           NaN  2000
7      1693           1         FL  ...           5.83           NaN  2001
8      1694           1         FL  ...           4.38           NaN  2001
9      1587           1         FL  ...           6.79           NaN  2000

[10 rows x 15 columns]
```

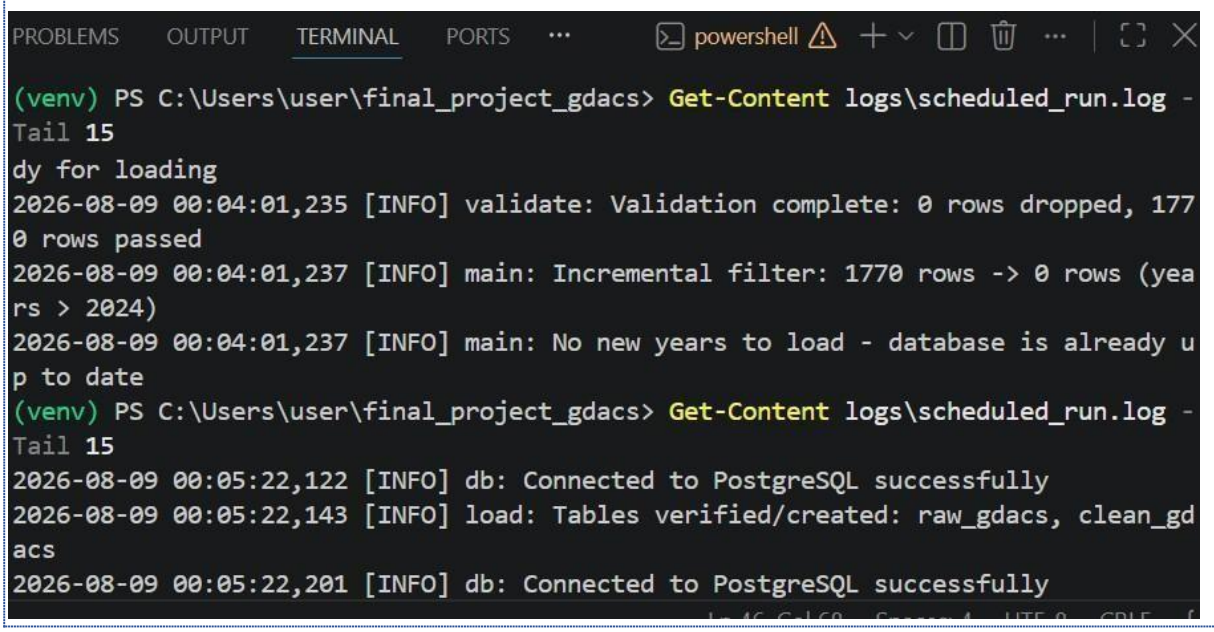
2.3 Data Layering (Raw and Cleaned Tables)

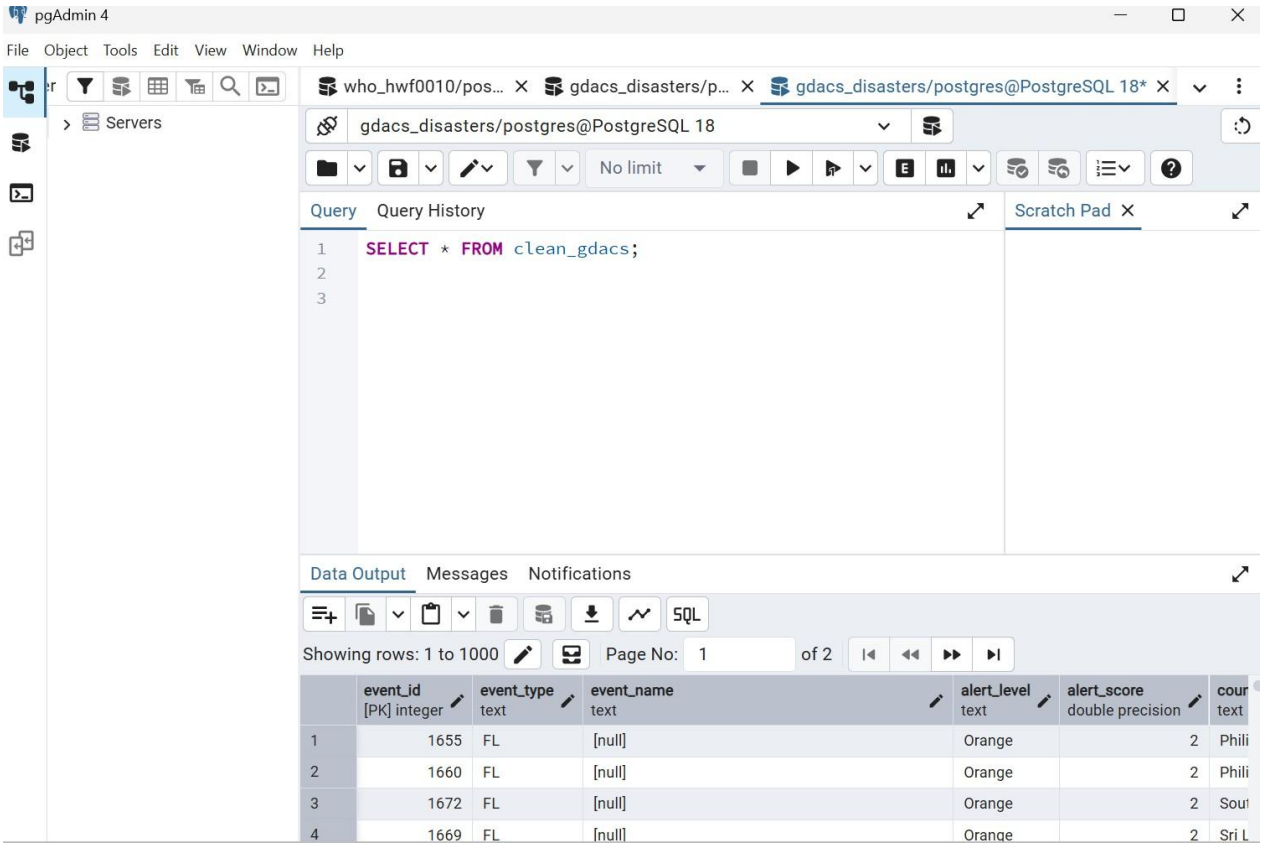
Instead of loading data straight into one final table, the project uses two layers in PostgreSQL, matching real data team practice:

- raw_gdacs: stores the untouched API response exactly as it came in, saved as JSON. This is a safety copy in case something goes wrong later in the pipeline.
  clean_gdacs: stores the cleaned, checked, ready-to-use data that Tableau and the machine learning models actually use.

Loading into clean_gdacs uses an upsert (insert new rows, or update existing ones if the event_id already exists). This means running the pipeline twice does not create duplicate rows — it safely updates existing data instead.

- 
pgAdmin 4
File Object Tools Edit View Window Help
Servers (2)
PostgreSQL 17
PostgreSQL 18
Databases (5)
gdacs_disasters
gdacs_project
Casts
Catalogs
Event Triggers
Extensions
Foreign Data W
Languages
Publications
Schemas
Subscriptions
postgres
stock_managame
who_hwf0010
Login/Group Roles
Tablespaces
gdacs_disasters/postgres@PostgreSQL 18*
No limit
Query
Query History
Scratch Pad
SELECT payload FROM raw_gdacs LIMIT 1;
Data Output
Messages
Notifications
SQL
Showing rows: 1 to 1
Page No: 1
of 1
payload
jsonb
{"type": "FeatureCollection", "features": [{"bbox": [123.132, 10.412, 123.132, 10.412], "type": "Feature", "geometry": {"type": "Point", "coo


PROBLEMS
OUTPUT
TERMINAL
PORTS
powershell
(venv) PS C:\Users\user\final_project_gdacs> Get-Content logs\scheduled_run.log -Tail 15
dy for loading
2026-08-09 00:04:01,235 [INFO] validate: Validation complete: 0 rows dropped, 1770 rows passed
2026-08-09 00:04:01,237 [INFO] main: Incremental filter: 1770 rows -> 0 rows (years > 2024)
2026-08-09 00:04:01,237 [INFO] main: No new years to load - database is already up to date
(venv) PS C:\Users\user\final_project_gdacs> Get-Content logs\scheduled_run.log -Tail 15
2026-08-09 00:05:22,122 [INFO] db: Connected to PostgreSQL successfully
2026-08-09 00:05:22,143 [INFO] load: Tables verified/created: raw_gdacs, clean_gdacs
2026-08-09 00:05:22,201 [INFO] db: Connected to PostgreSQL successfully


pgAdmin 4
File Object Tools Edit View Window Help
Servers
who_hwf0010/pos...
gdacs_disasters/p...
gdacs_disasters/postgres@PostgreSQL 18*
gdacs_disasters/postgres@PostgreSQL 18
No limit
Query
Query History
Scratch Pad
1 SELECT * FROM clean_gdacs;
2
3
Data Output
Messages
Notifications
SQL
Showing rows: 1 to 1000
Page No: 1
of 2
event_id [PK] integer
event_type text
event_name text
alert_level text
alert_score double precision
cour text
1 1655 FL [null] Orange 2 Phili
2 1660 FL [null] Orange 2 Phili
3 1672 FL [null] Orange 2 Sout
4 1669 FL [null] Orange 2 Sri L

3. Data Validation

Before any data moves from the raw table into the cleaned table, the pipeline checks it for problems. These checks are written directly in Python code (in validate.py), not left for the database to catch by accident:

- Missing values: rows missing an event ID, event type, alert level, year, latitude, or longitude are removed.
- Wrong data types: rows where year, latitude, or longitude are not proper numbers are removed.
- Values outside a sensible range: latitude must be between -90 and 90, longitude between -180 and 180, and year between 2000 and 2024.
- Unexpected categories: any row where alert_level is something other than 'Orange' or 'Red' is removed, since only those two levels are allowed in this project.

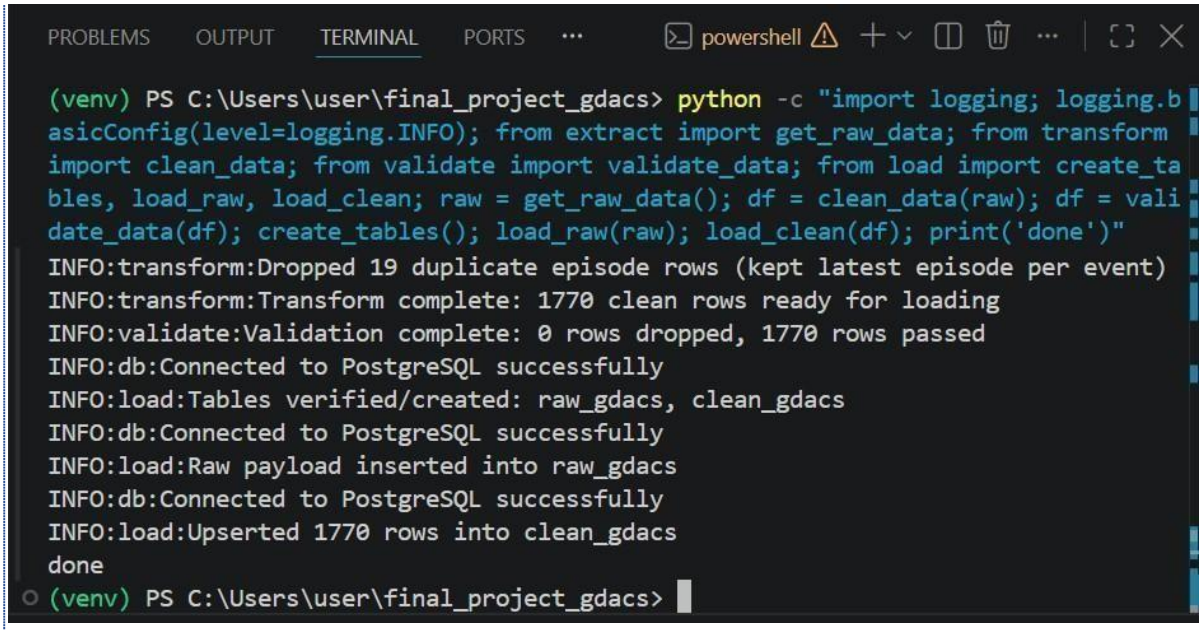
When the pipeline was run on the real GDACS data, these checks did not need to drop any rows — the data that made it past cleaning was already valid. This tells me that most of GDACS's data quality problems are structural (pagination, extra alert levels, missing fields) rather than individual bad values inside each record, which is why the validation step still matters even though it found nothing to reject this time.

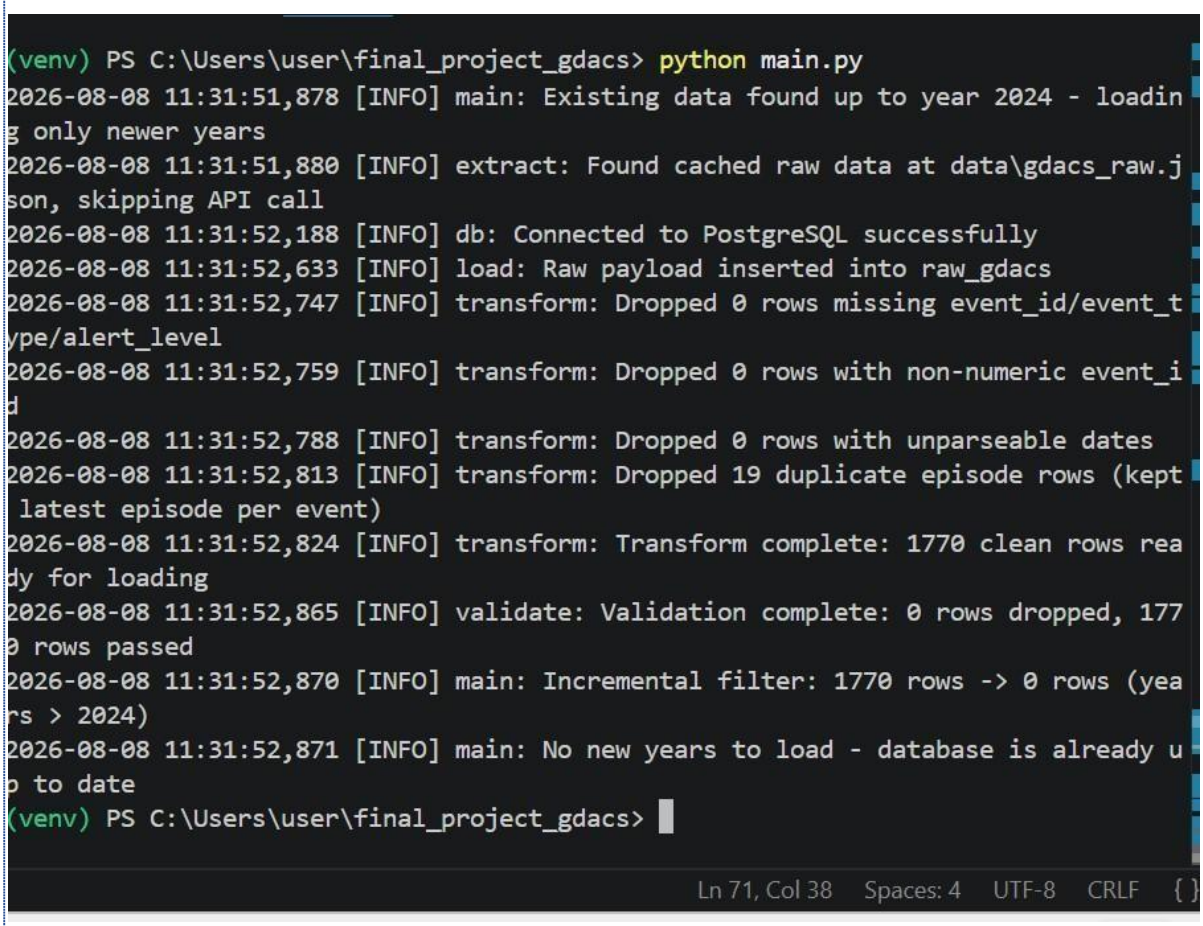
4. Edge Cases Handled

A number of real problems came up while building this pipeline, and each one is handled directly in the code instead of being ignored:

- API pagination limit: GDACS only returns 100 records per request. Handled by looping through pages until the data runs out.
- Green alert events excluded by default: GDACS's SEARCH endpoint quietly leaves out Green alerts unless you ask for them directly. I discovered this by testing the parameter explicitly, and made the decision to keep the project scoped to Orange/Red only.

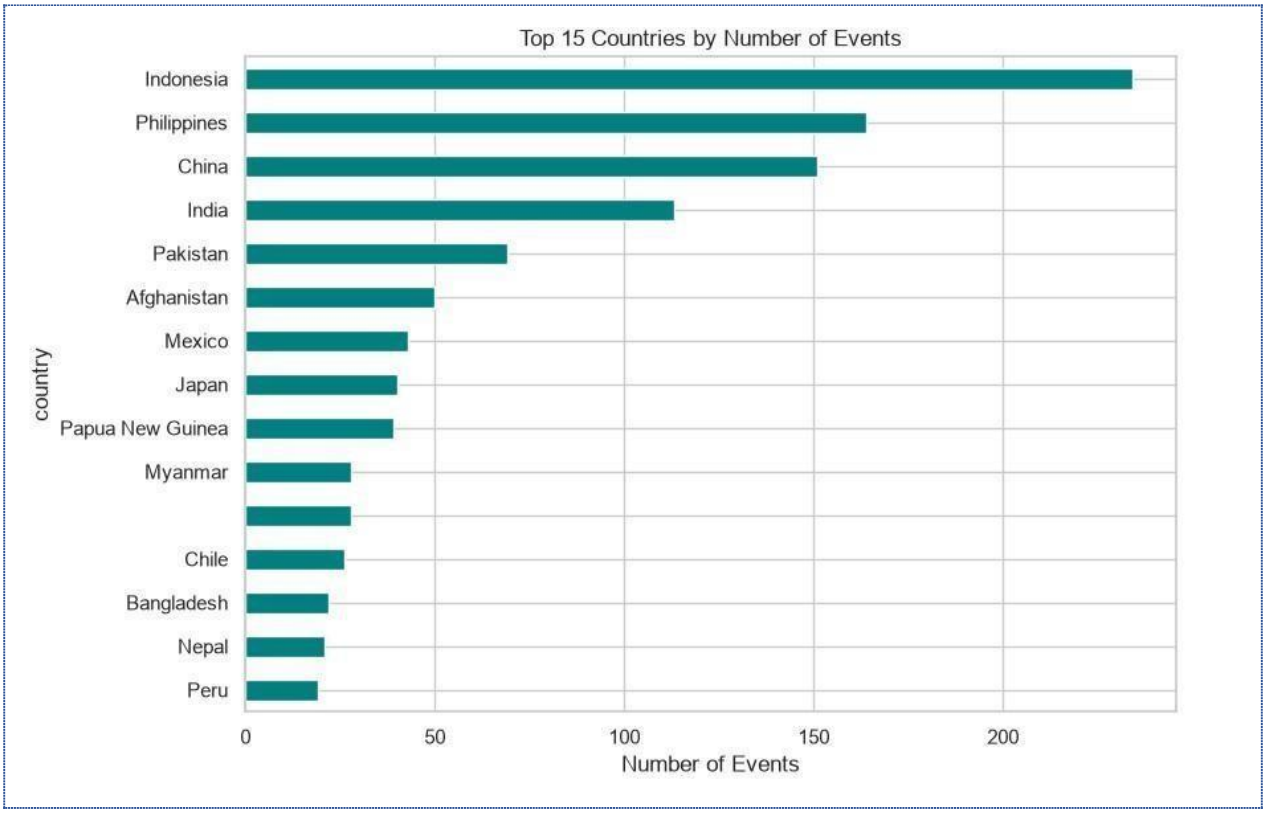
- Zero rows returned: if a request returns no events, the pipeline logs a warning instead of crashing.
- Database connection failure: if PostgreSQL is not reachable (wrong password, service not running), the pipeline gives a clear error message instead of a confusing crash.
- API unreachable: if the internet or the GDACS server is down, the pipeline retries with increasing wait times before failing with a clear message.
- Hidden 'NaN' text bug: missing text values were accidentally being saved as the word 'NaN' instead of being empty. Found by checking the exported CSV directly, then fixed.
- Hidden blank-space values: some fields (like iso3 for a couple of countries) contained only spaces instead of being truly empty. Found the same way, then fixed.
- Inconsistent country name spacing: some country names had extra spaces, which would have split one country into two separate categories in charts. Found and fixed before building the dashboard.

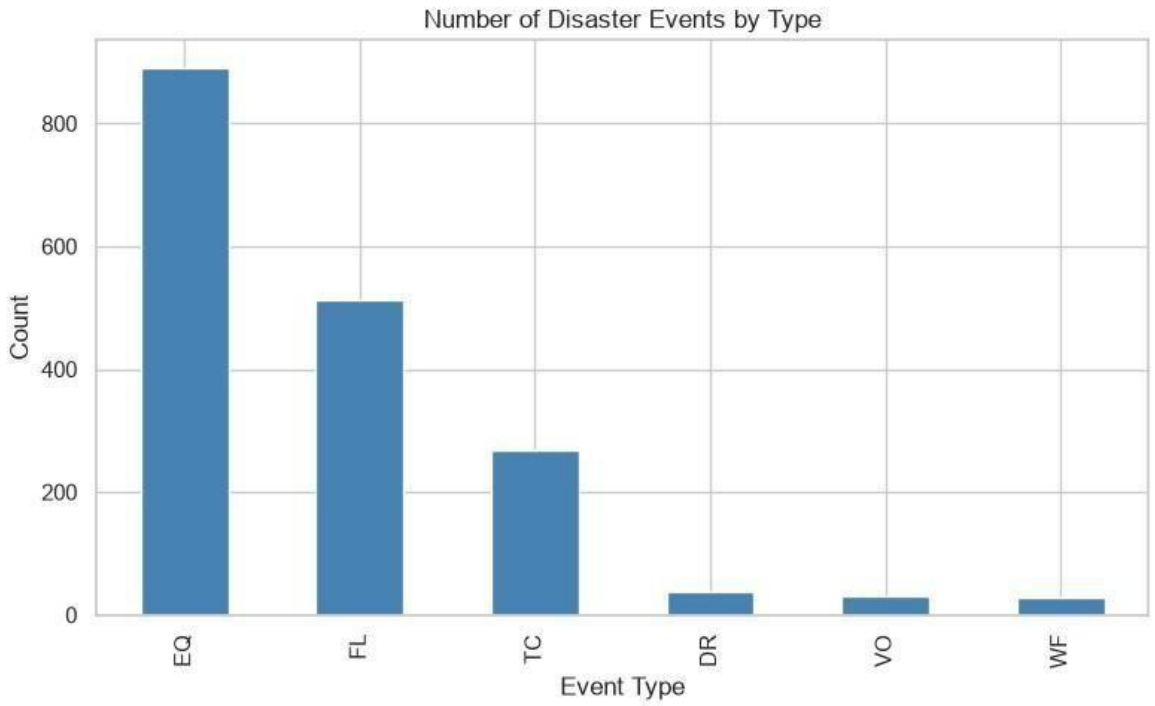

PROBLEMS
OUTPUT
TERMINAL
PORTS
powershell
(venv) PS C:\Users\user\final_project_gdacs> python -c "import logging; logging.basicConfig(level=logging.INFO); from extract import get_raw_data; from transform import clean_data; from validate import validate_data; from load import create_tables, load_raw, load_clean; raw = get_raw_data(); df = clean_data(raw); df = validate_data(df); create_tables(); load_raw(raw); load_clean(df); print('done')"
INFO:transform:Dropped 19 duplicate episode rows (kept latest episode per event)
INFO:transform:Transform complete: 1770 clean rows ready for loading
INFO:validate:Validation complete: 0 rows dropped, 1770 rows passed
INFO:db:Connected to PostgreSQL successfully
INFO:load:Tables verified/created: raw_gdacs, clean_gdacs
INFO:db:Connected to PostgreSQL successfully
INFO:load:Raw payload inserted into raw_gdacs
INFO:db:Connected to PostgreSQL successfully
INFO:load:Upserted 1770 rows into clean_gdacs
done
(venv) PS C:\Users\user\final_project_gdacs>

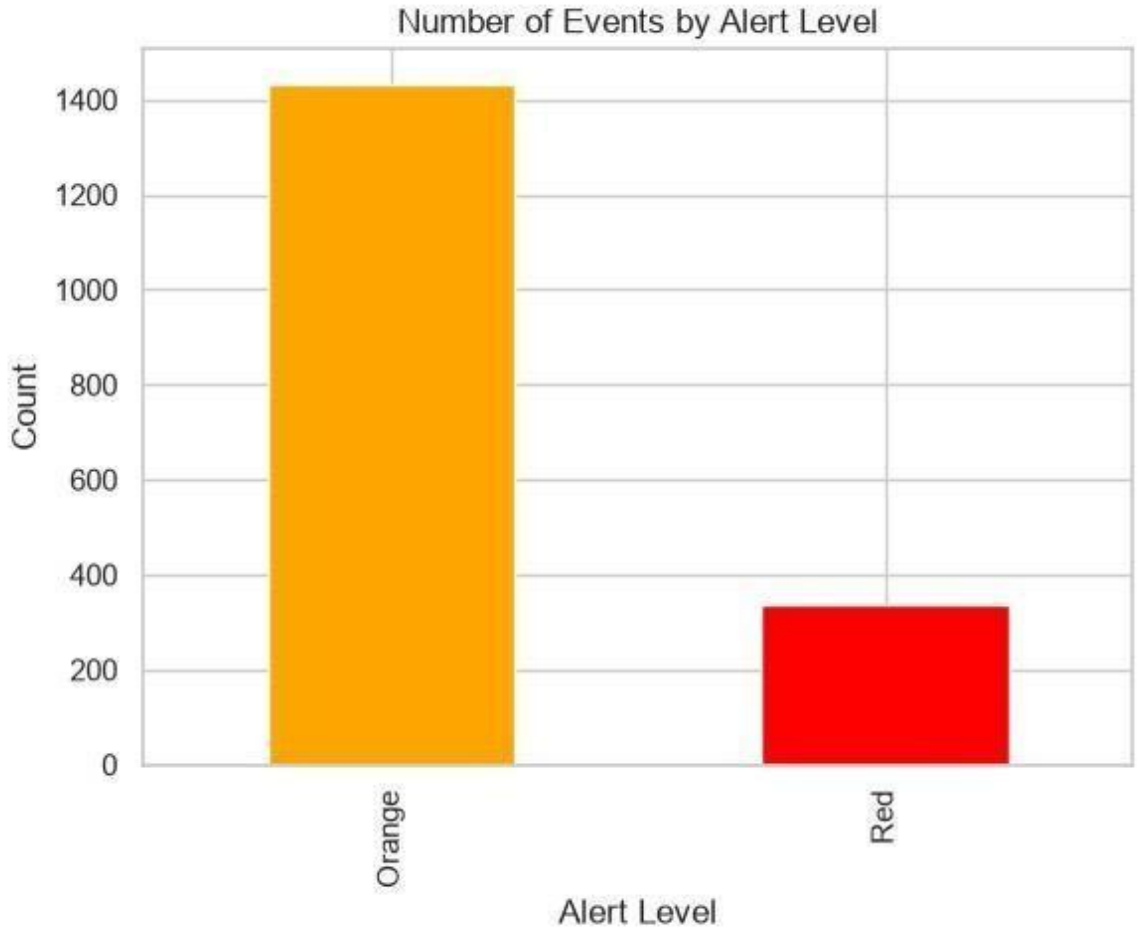

(venv) PS C:\Users\user\final_project_gdacs> python main.py
2026-08-08 11:31:51,878 [INFO] main: Existing data found up to year 2024 - loading only newer years
2026-08-08 11:31:51,880 [INFO] extract: Found cached raw data at data\gdacs_raw.json, skipping API call
2026-08-08 11:31:52,188 [INFO] db: Connected to PostgreSQL successfully
2026-08-08 11:31:52,633 [INFO] load: Raw payload inserted into raw_gdacs
2026-08-08 11:31:52,747 [INFO] transform: Dropped 0 rows missing event_id/event_type/alert_level
2026-08-08 11:31:52,759 [INFO] transform: Dropped 0 rows with non-numeric event_id
2026-08-08 11:31:52,788 [INFO] transform: Dropped 0 rows with unparseable dates
2026-08-08 11:31:52,813 [INFO] transform: Dropped 19 duplicate episode rows (kept latest episode per event)
2026-08-08 11:31:52,824 [INFO] transform: Transform complete: 1770 clean rows ready for loading
2026-08-08 11:31:52,865 [INFO] validate: Validation complete: 0 rows dropped, 1770 rows passed
2026-08-08 11:31:52,870 [INFO] main: Incremental filter: 1770 rows -> 0 rows (years > 2024)
2026-08-08 11:31:52,871 [INFO] main: No new years to load - database is already up to date
(venv) PS C:\Users\user\final_project_gdacs>
Ln 71, Col 38
Spaces: 4
UTF-8
CRLF
{ }

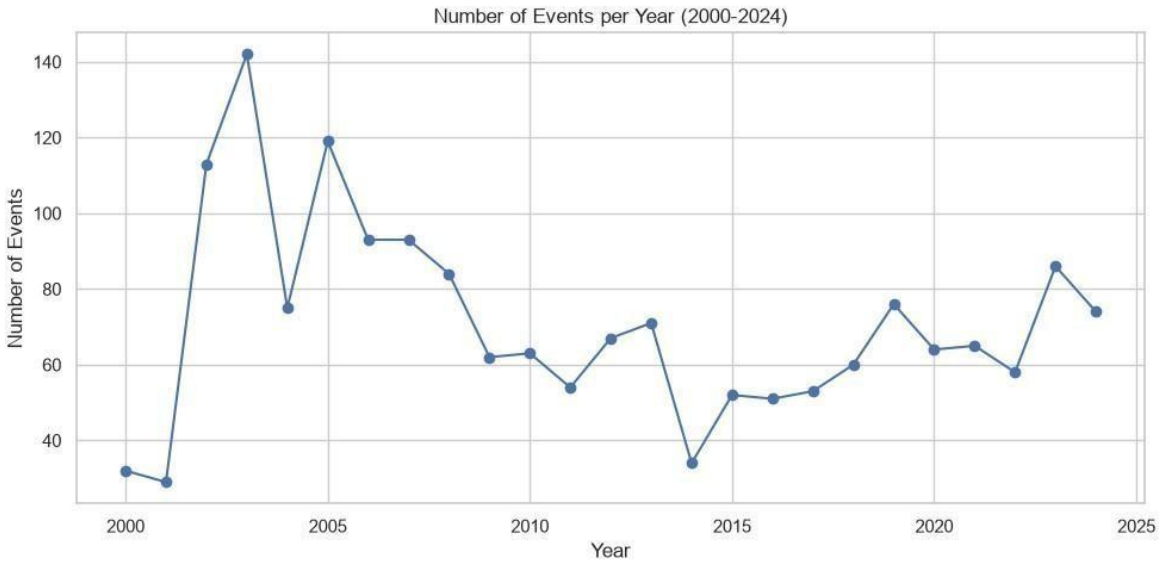
5. Exploratory Data Analysis (EDA)

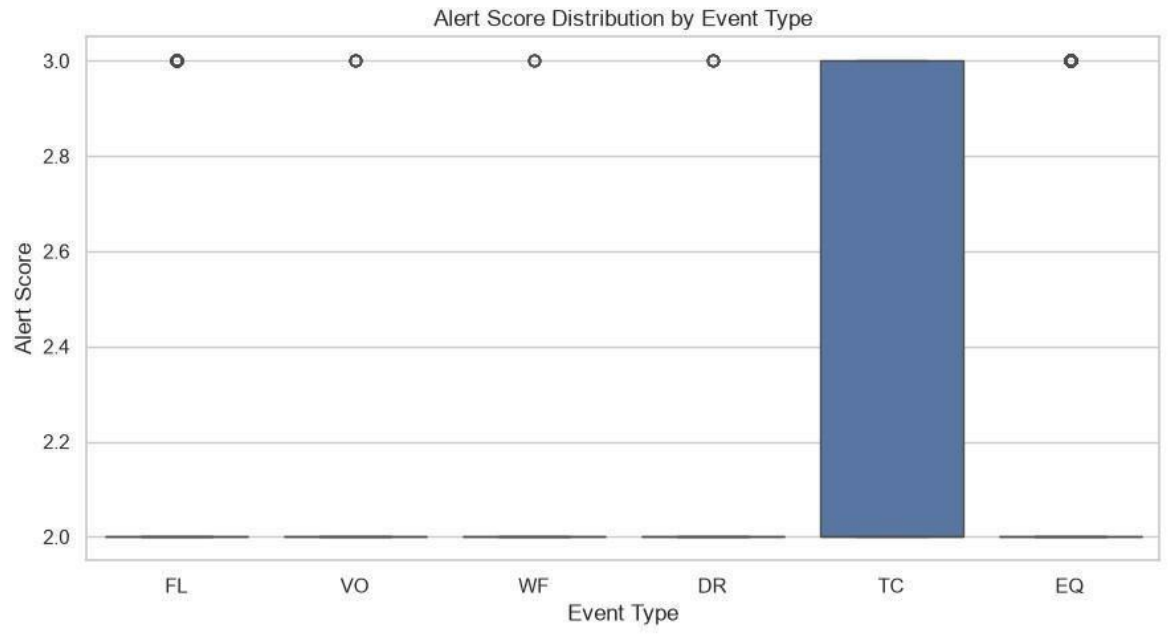
Before building any models, I looked at the cleaned dataset as a whole using a Python script (eda_report.py), which prints a summary and saves six charts as image files.

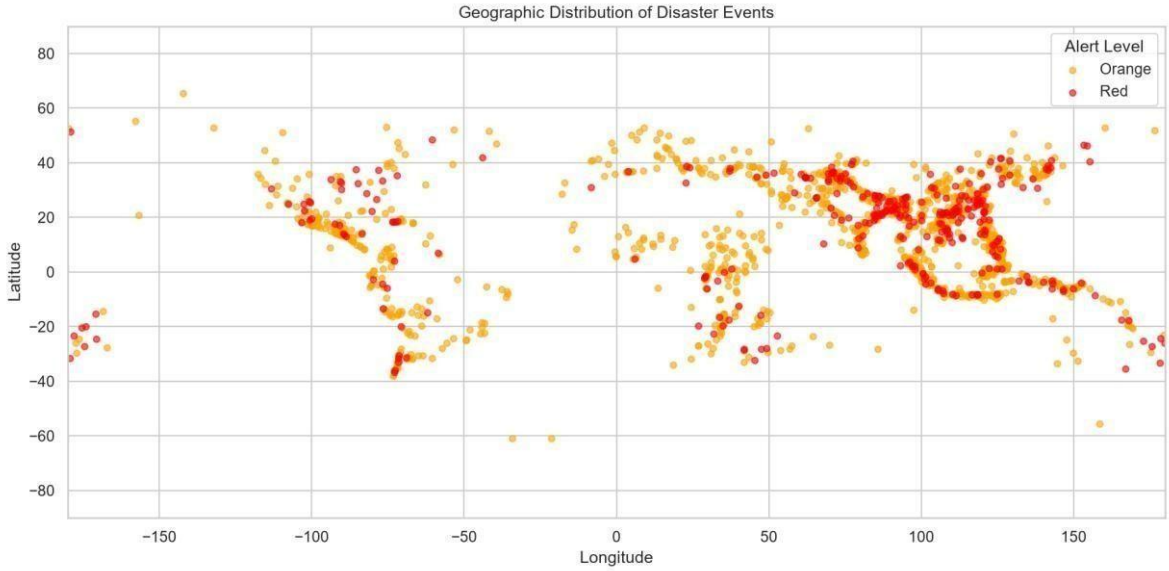



Number of Disaster Events by Type
Count
800
600
400
200
0
EQ
FL
TC
DR
VO
WF
Event Type


Number of Events by Alert Level
1400
1200
1000
800
600
400
200
0
Count
Orange
Red
Alert Level


Number of Events per Year (2000-2024)
140
120
100
80
60
40
Number of Events
2000
2005
2010
2015
2020
2025
Year





From these charts, earthquakes (EQ) and floods (FL) are by far the most common event types in the dataset, followed by tropical cyclones. The number of recorded events was highest in the early-to-mid 2000s and has generally been lower and more stable in recent years, which may reflect changes in how GDACS records events rather than fewer real disasters happening. The geographic scatter plot roughly traces the shape of the world's continents just from event locations, and shows clear clustering along

known disaster-prone areas such as the Pacific 'Ring of Fire' for earthquakes and the tropics for cyclones and floods. Looking at alert score by event type, certain event types such as tropical cyclones tend to have a wider spread of alert scores than others, suggesting their severity varies more from event to event.

6. Machine Learning

6.1 Approach

The goal was to predict alert_level (Orange or Red) for each disaster event — a classification problem. The features used were: severity_value (a measure of how strong the event was), latitude, longitude, and the event type (turned into separate yes/no columns, one per disaster type).

Two features were deliberately left out, for good reasons found during testing:

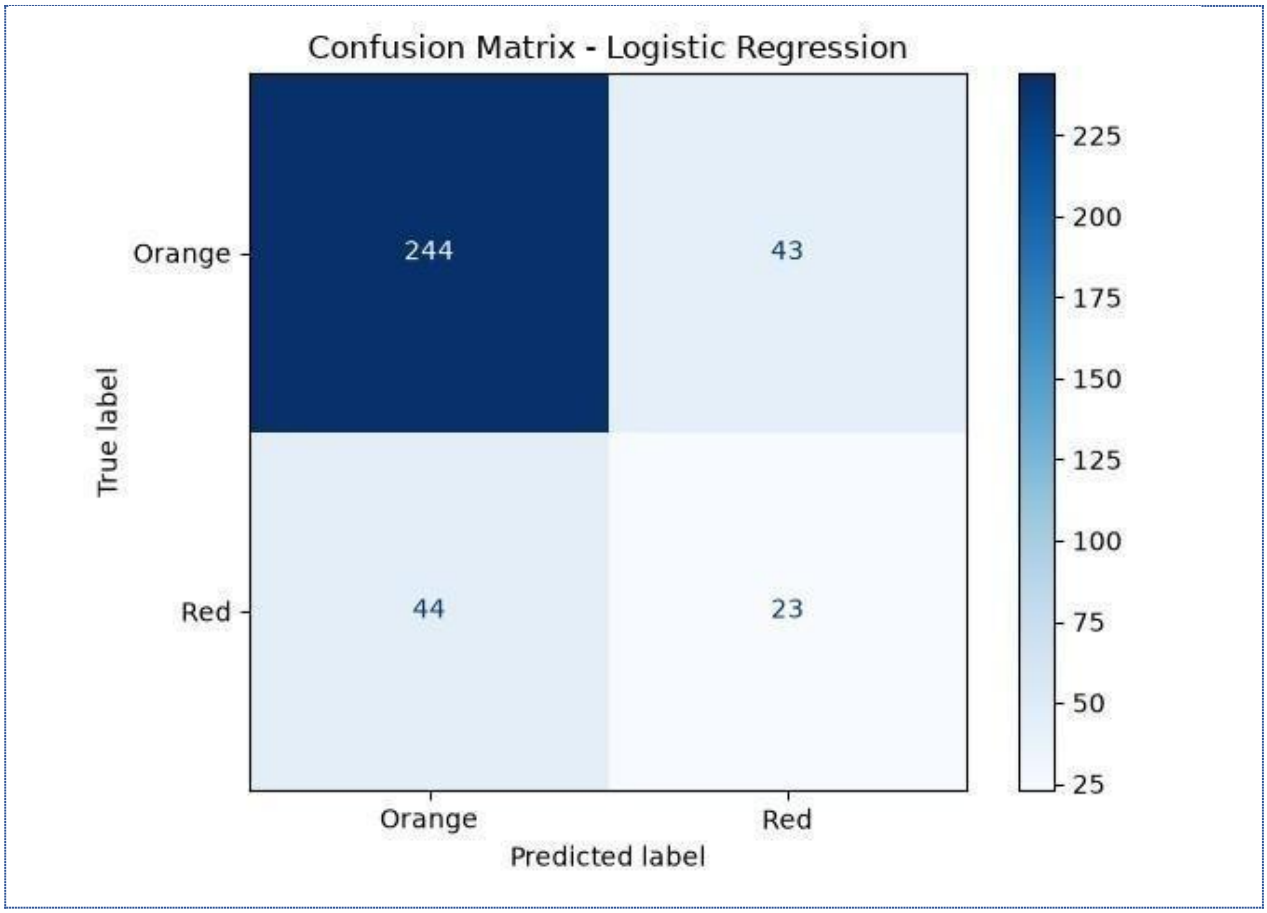
- alert_score was removed because GDACS calculates alert_level directly from alert_score. Including it let both models score a suspicious 100 percent accuracy, which is a sign of data leakage rather than real learning. Removing it forced the models to learn from genuinely independent information.
- population_value was removed because, as explained in Section 2.2, GDACS's SEARCH endpoint never actually provides this field.

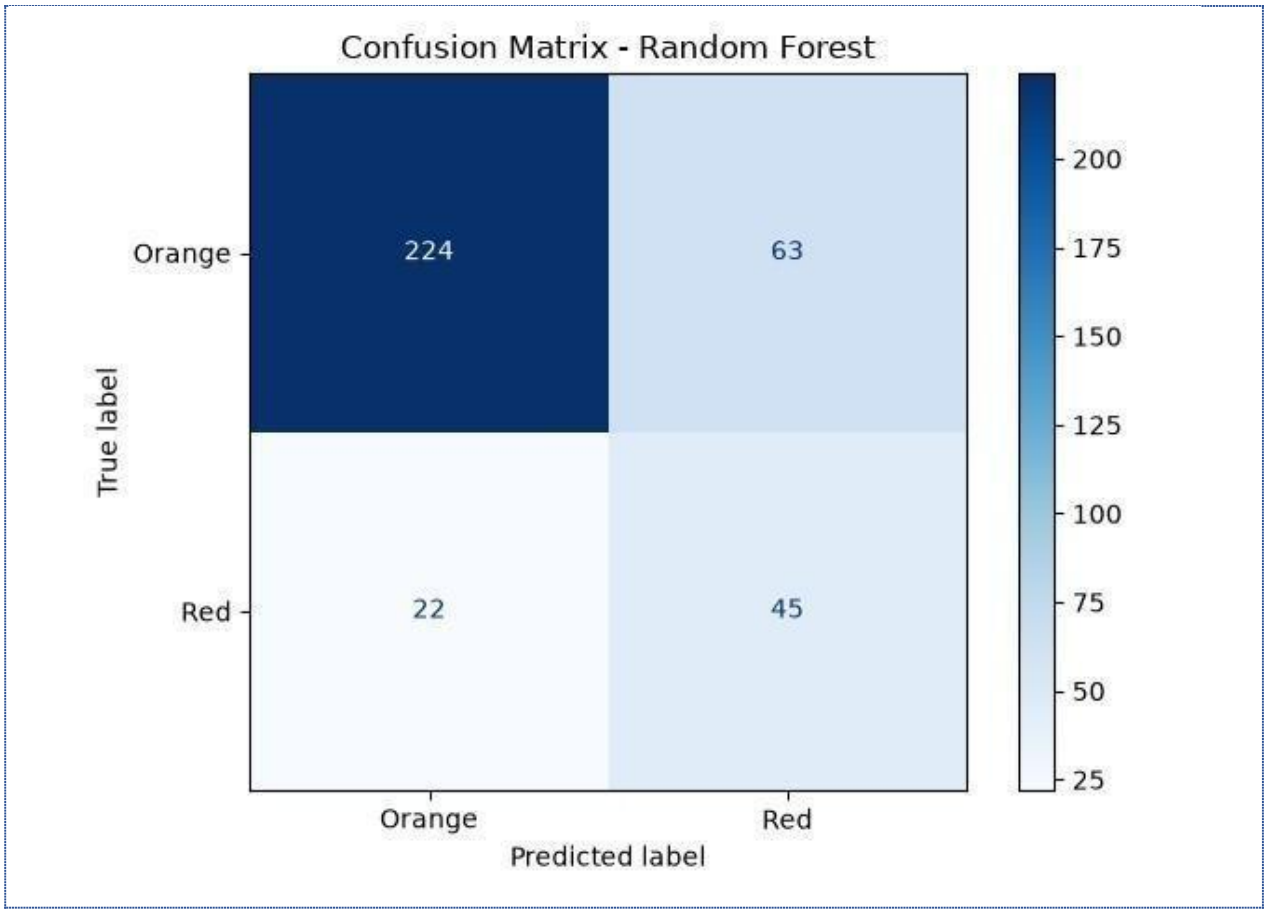
The data was split into 80 percent for training and 20 percent for testing, using a stratified split so that both the training and test sets kept the same roughly 4-to-1 ratio of Orange to Red events found in the full dataset. Testing was always done on the 20 percent the model never saw during training.

6.2 Model Comparison

Two models were trained and compared: Logistic Regression (a simple, straight-line model) and Random Forest (a model built from many decision trees).

Model	Test Accuracy	Precision (Red)	Recall (Red)	F1 (Red)
Logistic Regression	0.777	0.393	0.328	0.358
Random Forest	0.734	0.378	0.627	0.472


Confusion Matrix - Logistic Regression
Orange
Red
True label
244
43
44
23
Orange
Red
Predicted label
225
200
175
150
125
100
75
50
25



6.3 Checking for Overfitting and Underfitting

To check for overfitting, Random Forest was trained several times using different max_depth settings (how deep each decision tree is allowed to grow), comparing training accuracy against test accuracy each time:

max_depth	Train Accuracy	Test Accuracy
2	0.810	0.811
4	0.844	0.842
6	0.871	0.833
10	0.961	0.828

None (no limit)	1.000	0.839

At max_depth=2, the model is likely underfitting: it is too simple, so training and test accuracy are both around 0.81, without much room to learn real patterns. As depth increases to 4 and 6, training accuracy climbs while test accuracy stays close, which is a healthy sign. Past depth 6, training accuracy keeps rising sharply (up to a perfect 1.000 with no depth limit) while test accuracy barely changes or slightly drops, which is a clear overfitting pattern: the model is memorizing the training data rather than learning something that generalizes. Based on this, max_depth=6 was kept as the final setting, since it gives a good balance between learning enough and not memorizing too much.

6.4 Which Model Did Better, and Why

Looking at accuracy alone, Logistic Regression appears slightly better (0.777 vs 0.734). But accuracy is misleading here because the dataset has far more Orange events than Red events (roughly 4 to 1). A model can score well just by guessing Orange most of the time.

The better way to judge these models is by how well they catch real Red alerts. Random Forest correctly identifies 62.7 percent of actual Red events (recall), compared to only 32.8 percent for Logistic Regression. Random Forest's F1 score for Red (0.472) is also notably higher than Logistic Regression's (0.358). This means Random Forest is meaningfully better at the part of the problem that actually matters: correctly flagging the more serious, less common disasters. Random Forest performs better here because it can pick up on non-linear patterns and interactions between event type, location, and severity, while Logistic Regression can only draw a straight-line boundary between the two classes, which is too simple for this problem. Because of this, Random Forest is the model I would choose as the better one for this project, even though its raw accuracy number is slightly lower.

7. Tableau Dashboard

The dashboard was built in Tableau Public, connecting to the cleaned data exported as a CSV file, plus a second file containing the Random Forest model's predictions on the test set.

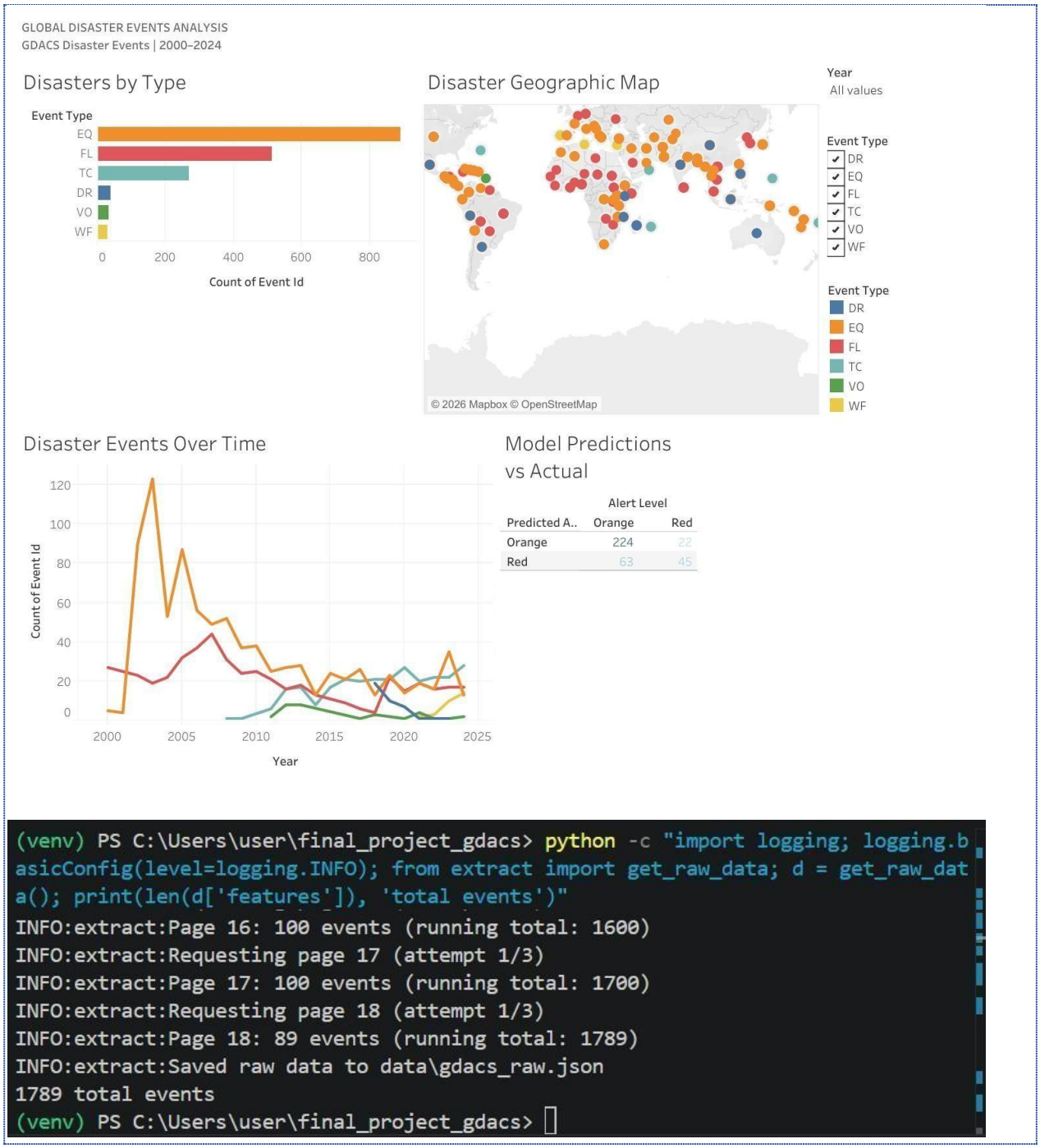
Published dashboard link:
https://public.tableau.com/app/profile/nkuranga.baziga.caleb/viz/Global_Disaster_Events_Analysis/Dashboard1

The dashboard has four charts:

- Disasters by Type — a bar chart showing how many events of each disaster type are in the dataset
- Disaster Geographic Map — a world map showing where events happened, colored by disaster type

- Disaster Events Over Time — a line chart showing the number of events per year, split by alert level
- Model Predictions vs Actual — a grid comparing what the Random Forest model predicted against the real alert level, built directly from the model's test set output

Two interactive filters are included: Year and Event Type. Both apply across the dashboard, so changing either one updates the relevant charts at the same time.




```
(venv) PS C:\Users\user\final_project_gdacs> python -c "import logging; logging.b
asicConfig(level=logging.INFO); from extract import get_raw_data; d = get_raw_dat
a(); print(len(d['features']), 'total events')"
INFO:extract:Page 16: 100 events (running total: 1600)
INFO:extract:Requesting page 17 (attempt 1/3)
INFO:extract:Page 17: 100 events (running total: 1700)
INFO:extract:Requesting page 18 (attempt 1/3)
INFO:extract:Page 18: 89 events (running total: 1789)
INFO:extract:Saved raw data to data\gdacs_raw.json
1789 total events
(venv) PS C:\Users\user\final_project_gdacs>
```

8. AI Assistant

The AI assistant is a small Python program (ai_assistant.py) that lets someone type a plain-English question about the disaster data and get a plain-English answer back. It uses the Groq API with the llama-3.3-70b-versatile model, which is free and does not need a credit card.

It works in three steps:

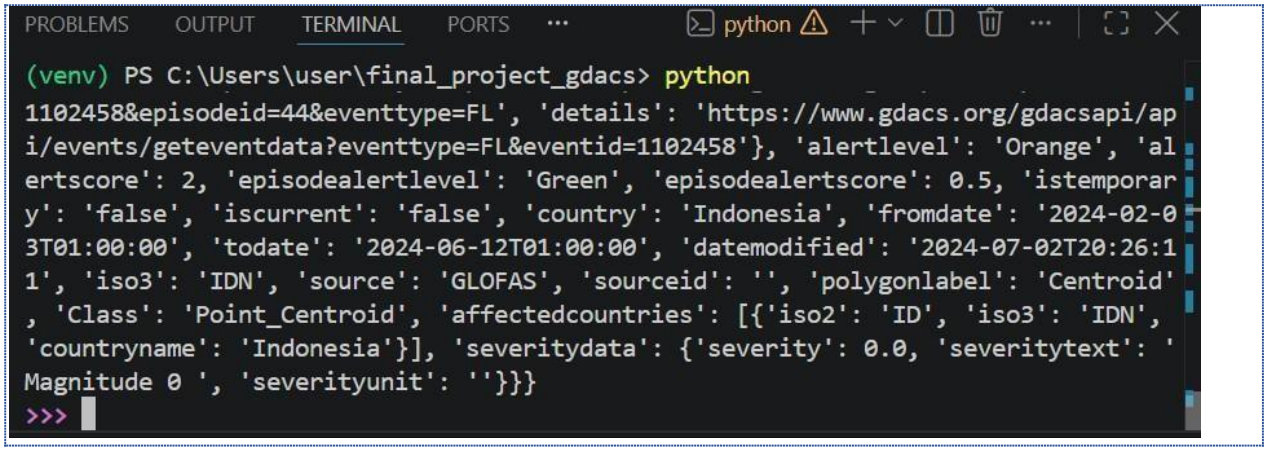
- The user's question is sent to Groq along with a description of the clean_gdacs table, asking it to write a single SQL SELECT query that would answer the question.
- The program runs that SQL query against the PostgreSQL database and gets the result back.
- The result is sent back to Groq a second time, asking it to turn the raw result into a natural, plain-English sentence.

A few safety measures are built in to handle things going wrong:

- Only SELECT queries are ever allowed to run. If the AI ever generated something like DROP, DELETE, or UPDATE, the program refuses to run it and tells the user instead of risking the database.
- If the SQL query fails to run (for example a wrong column name), the program catches the error and gives the user a friendly message instead of crashing.
- If the question cannot be answered from this dataset at all (for example asking about tomorrow's weather), the AI is told to reply with a fixed marker, and the program then tells the user plainly that it cannot answer that from this data.

Example of the assistant working correctly: when asked 'Which country had the most Red alert events?', the assistant generated a SQL query grouping by country and counting Red-level events, ran it against the database, and replied with the correct country by name in a natural sentence.

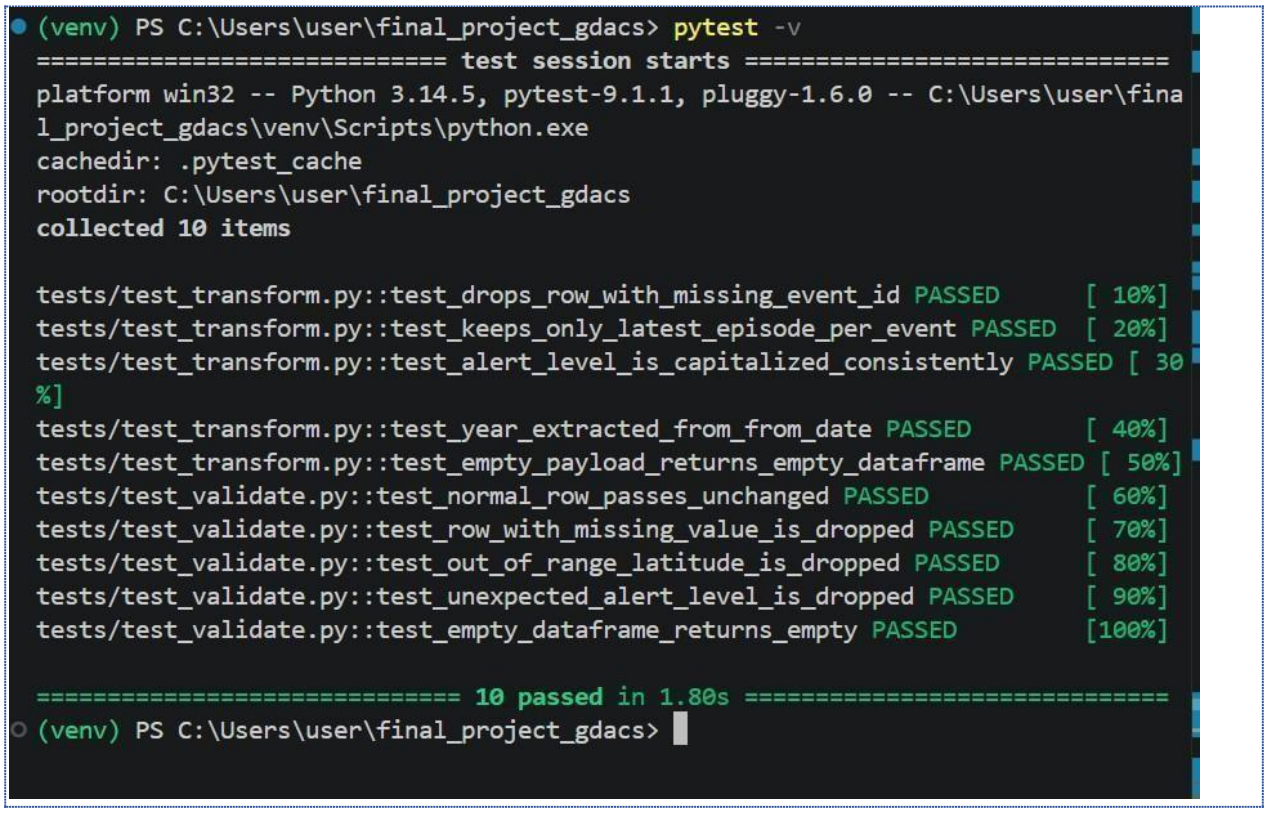
Example of a confusing question handled gracefully: when asked 'What was the average population exposed for floods in Kenya?', the SQL query ran without error, but returned no matching data (since GDACS does not provide population data, as explained in Section 2.2). Instead of making up a number or crashing, the assistant correctly replied that this information was not available. When asked something fully outside the dataset, like 'What's the weather like tomorrow?', the assistant recognized it could not be answered from the disaster data and said so directly, instead of guessing.



9. Testing

Automated tests were written using pytest, covering three parts of the project:

- tests/test_transform.py — checks that rows with missing data are dropped, duplicate episodes of the same event are correctly reduced to one row, alert level text is standardized, and an empty API response does not break the cleaning function.
- tests/test_validate.py — checks that a normal row passes through unchanged, a row with a missing value is dropped, a row with an out-of-range value is dropped, and an unexpected alert level is dropped.
- tests/test_ai_assistant.py — checks that a normal SELECT query is accepted, that DROP/DELETE/UPDATE queries are correctly rejected for safety, and that the assistant responds with a clear message instead of crashing when the AI fails to generate SQL or when a question cannot be answered.



While building the pipeline, debugging mostly happened by checking the output of each stage by hand before trusting it: printing row counts after each cleaning step, manually inspecting a few rows in pgAdmin after every load, and comparing numbers (like total event counts or model accuracy) across repeated runs to make sure they matched expectations. This approach is exactly what caught the three data bugs

described in Section 2.2 (the NaN text bug, the whitespace-only values, and the inconsistent country spacing) — none of these would have been obvious just from reading the code, only from checking the actual data that came out of it. If a future run produced unexpected results, the first thing I would check is the pipeline log file (logs/pipeline.log) for warnings about dropped rows, followed by a direct SELECT query in pgAdmin to compare the raw and cleaned tables.

10. Resume and Incremental Loading

The pipeline can stop partway through and pick back up without starting over, in two ways:

- Resume after extraction: the raw API response is saved to a local file (data/gdacs_raw.json) right after a successful download. If the pipeline is interrupted before loading finishes and then restarted, it checks for this file first and reuses it instead of calling the API again.
- Incremental loading: before loading new data, the pipeline checks the most recent year already stored in clean_gdacs. On later runs, only rows for years newer than that are loaded, instead of reprocessing everything from 2000 again.

The main limitation of this approach is that it only adds new years going forward — it does not go back and check whether GDACS has revised or corrected details on an event from a year that is already loaded. For a project of this size, this was an acceptable trade-off, but a more complete version would periodically re-check recent years rather than only ever moving forward.

11. Data Dictionary

Column	Type	Meaning	Source
event_id	integer	Unique ID for each disaster event	GDACS eventid
event_type	text	Disaster type code (EQ, FL, TC, DR, VO, WF)	GDACS eventtype
event_name	text	Name of the event, if GDACS gave it one (often blank for smaller events)	GDACS eventname
alert_level	text	Orange or Red (Green excluded, see Section 1)	GDACS alertlevel

alert_score	numeric	GDACS's internal severity score (not used as a model feature, see Section 6.1)	GDACS alertscore
country	text	Country or region name, cleaned of extra spaces	GDACS country
iso3	text	3-letter country code, used for the Tableau map	GDACS iso3
from_date / to_date	timestamp	Event start and end date	GDACS fromdate / todate
year	integer	Year taken from from_date	Calculated during cleaning
latitude / longitude	numeric	Event location coordinates	GDACS geometry.coordinates
severity_value	numeric	A measure of how strong the event was (meaning depends on event type)	GDACS severitydata.severity
severity_unit	text	Unit for severity_value (for example M for magnitude, ha for hectares)	GDACS severitydata.severityunit
loaded_at	timestamp	When this row was saved or last updated in the database	Added by the pipeline

12. What I Would Improve With More Time

- Incremental loading only adds new years and never re-checks a year that is already loaded, even if GDACS later revises that year's data. A more complete version would periodically re-check recent years.
- population_value looked like a useful feature at the start but turned out to be unavailable from the SEARCH endpoint. A future version could call GDACS's per-event endpoint to try to get this data, though that would mean hundreds of extra API calls.

- The class imbalance between Orange and Red events (roughly 4 to 1) limits how well any model can learn to recognize Red events. Trying techniques like oversampling the Red class, or adjusting the model's decision threshold, could improve Red recall further.
- The AI assistant answers each question independently and has no memory of earlier questions in the same conversation, so it cannot handle follow-up questions like 'what about the lowest one instead.'
- The Random Forest's max_depth was chosen from a small manual test of five values. A proper grid search across more settings and more parameters (like n_estimators) could find an even better setting.
- The Tableau dashboard is built from a static CSV export, so it does not automatically update when new data is loaded into PostgreSQL — new data would need to be re-exported and the dashboard republished.

Conclusion

In conclusion, this project successfully analyzed global disaster events using data from GDACS. The data was collected, cleaned, validated, and stored in PostgreSQL before being used for analysis, machine learning, and visualization. The project also handled important data problems such as missing values, duplicate events, API limits, and incorrect country names.

The analysis showed useful patterns in disaster types, locations, and alert levels. Two machine learning models were tested, and Random Forest was selected because it was better at identifying serious Red alert events. The Tableau dashboard also made the results easier to understand through charts, maps, and filters.

Overall, this project showed how Big Data tools and techniques can be used to turn raw disaster data into useful information for analysis and decision-making.